

LAB 6 · WEEKS 11–12**Object Storage Security & the Data Security Lifecycle**

Bucket exposure, resource policies, SSE-KMS, versioning and provable deletion — Amazon S3 on LocalStack

Lab Learning Outcomes

At the end of this lab, you will be able to:

- Provision object storage, classify the data you place in it, and explain why object storage has a different security model from block or file storage.
- Reproduce the **archetypal cloud breach** — a publicly readable bucket — and remediate it with Block Public Access and a least-privilege bucket policy.
- Distinguish **identity-based** (IAM) from **resource-based** (bucket policy) authorisation, and predict the outcome when the two disagree.
- Enforce **encryption at rest** with SSE-KMS and reason about a policy that mandates encryption on upload.
- Issue **time-bounded delegated access** with a presigned URL and assess its risks.
- Apply **versioning, lifecycle and retention** rules, demonstrate object-level data remanence, and achieve provable deletion through cryptographic erasure.

Course & Assessment Mapping

Item	Mapping
Course Learning Outcome	CLO2 — Construct secure cloud operations that safeguard data confidentiality and integrity (VBE3)
Lecture topics	Week 4 (Data Protection) · Week 10 (Policy, Compliance & Risk) · Week 11 (Compliance Assessment & Reporting)
Value / skill clusters	VBE3 (Integrity) · SC8 (Integrated Problem-Solving)
CSA CCSK v5 domains	Domain 5 (Data Security) · Domain 4 (Organisation Management) · Domain 9 (Application Security — resource policy)
Assessment	Lab report (outputs + short answers) — contributes to the Lab Assignment

Lab Arrangement (2 Sessions over 2 Weeks)

Session	Week	Focus
Session A	Week 11	Object storage, data classification, the public-bucket breach and its remediation, identity vs resource policy (Tasks 1–4)
Session B	Week 12	Default encryption, presigned URLs, versioning and remanence, lifecycle and cryptographic erasure (Tasks 5–8), then the report

Note: Session A is about **who can reach the data**. Session B is about **what state the data is in** — encrypted, versioned, retained, or provably destroyed. Together they trace the full data security lifecycle from Week 4.

Technical Prerequisites

- A laptop with Docker Desktop / Docker Engine and a terminal.
- AWS CLI v2, pointed at LocalStack exactly as in Lab 1.
- A LocalStack auth token configured as in Lab 0.1 (the Resource Browser makes bucket state much easier to see).
- The KMS concepts from Lab 3 — this lab reuses envelope encryption and cryptographic erasure at bucket scale.
- `curl` for anonymous requests (pre-installed on macOS/Linux; use Git Bash or WSL on Windows).

Security tip: Object storage is the single most common source of real-world cloud data leaks. Every task in Session A corresponds to a control that, when missing, has produced a headline breach. Read the failures as carefully as the successes.

Session A (Week 11) — Object Storage & the Exposure Problem

One-Time Environment Setup

Start a clean, activated LocalStack instance and point the CLI at it. `ENFORCE_IAM=1` asks LocalStack to actually evaluate IAM policies rather than allowing everything — you need this for Task 4.

```
# Start clean
docker rm -f localstack 2>/dev/null

docker run -d --name localstack -p 4566:4566 \
  -e LOCALSTACK_AUTH_TOKEN=$LOCALSTACK_AUTH_TOKEN \
  -e ENFORCE_IAM=1 \
  localstack/localstack-pro:latest

# Point the CLI at LocalStack (repeat in every new terminal)
export EP='--endpoint-url=http://localhost:4566'
aws configure set aws_access_key_id test
aws configure set aws_secret_access_key test
aws configure set region us-east-1

aws $EP sts get-caller-identity
```

A successful call returns the LocalStack dummy identity (account `000000000000`). Record that account number — you will need it in the ARNs you write in Task 4.

Task 1 — Classify the Data Before You Store It

Security decisions follow classification, not the other way round. Create a bucket for a hospital records system and store three objects of different sensitivity, tagging each with its classification.

```
export BUCKET=miit-patient-records-$RANDOM
echo $BUCKET # write this down - you need it all lab
```

```
aws $EP s3api create-bucket --bucket $BUCKET

echo 'Ward visiting hours 10am-8pm' > public-notice.txt
echo 'Staff duty schedule, week 12' > internal-roster.txt
echo 'Patient: Ahmad bin Ali, Diagnosis: confidential' > confidential-record.txt

aws $EP s3api put-object --bucket $BUCKET --key public/notice.txt \
  --body public-notice.txt --tagging 'classification=public'
aws $EP s3api put-object --bucket $BUCKET --key internal-roster.txt \
  --body internal-roster.txt --tagging 'classification=internal'
aws $EP s3api put-object --bucket $BUCKET --key confidential/record.txt \
  --body confidential-record.txt --tagging 'classification=confidential'

aws $EP s3api list-objects-v2 --bucket $BUCKET \
  --query 'Contents[][Key,Size]' --output table
aws $EP s3api get-object-tagging --bucket $BUCKET --key confidential/record.txt
```

Complete this table in your report. The final column is the control you will actually build later in the lab.

Classification	Who may read it	Impact if leaked	Control you will apply
public			
internal			
confidential			

Note: Note the key names. A prefix such as `confidential/` is not a folder — object storage has a flat namespace and the slash is just part of the key. That matters because policies grant access by **key prefix**, which is why a sloppy prefix such as `*` exposes everything at once.

Task 2 — Reproduce the Archetypal Breach

Almost every 'exposed cloud storage' headline reduces to one resource policy that names "Principal": `"*"`. Build that policy deliberately, then read your own confidential record with no credentials at all.

```
cat > public-policy.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadEverything",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::$BUCKET/*"
  }]
}
JSON

aws $EP s3api put-bucket-policy --bucket $BUCKET --policy file://public-policy.json
aws $EP s3api get-bucket-policy --bucket $BUCKET --query Policy --output text
```

```
# The attacker's view: no AWS credentials, no CLI, just a URL
curl -s -o leaked.txt -w 'HTTP %{http_code}\n' \
  http://localhost:4566/$BUCKET/confidential/record.txt
cat leaked.txt
```

Caution: HTTP 200 and the patient record printed in your terminal is the whole breach. There was no exploit, no malware and no vulnerability — only a policy that said Principal: *. In your report, state which single word in the JSON caused the exposure.

Task 3 — Remediate with Block Public Access

Removing the bad policy fixes today's mistake. **Block Public Access** is the guardrail that stops tomorrow's — it overrides any policy or ACL that would make the bucket public, so a careless colleague cannot re-open it.

```
# 1. Remove the offending policy
aws $EP s3api delete-bucket-policy --bucket $BUCKET

# 2. Apply the account-level guardrail to the bucket
aws $EP s3api put-public-access-block --bucket $BUCKET \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

aws $EP s3api get-public-access-block --bucket $BUCKET

# 3. Try to re-introduce the public policy - the guardrail should refuse it
aws $EP s3api put-bucket-policy --bucket $BUCKET --policy file://public-policy.json

# 4. Re-test the anonymous read
curl -s -o /dev/null -w 'anonymous read now: HTTP %{http_code}\n' \
  http://localhost:4566/$BUCKET/confidential/record.txt
```

Verify or explain: LocalStack stores the Block Public Access configuration faithfully but does not always *enforce* it, so step 3 may succeed and step 4 may still return HTTP 200. If so, capture the get-public-access-block output as your evidence and state in your report (a) which of the four flags would have rejected the policy on real AWS, and (b) why a preventative guardrail is stronger than a detective control that merely reports the bucket as public.

Now write the policy you *should* have had: read access for your own account only, scoped to the internal prefix.

```
cat > least-privilege-policy.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AccountReadInternalOnly",
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::000000000000:root"},
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::$BUCKET/internal/*"
  }]
}
```

```

}
JSON

aws $EP s3api put-bucket-policy --bucket $BUCKET --policy file://least-privilege-policy.json
aws $EP s3api get-bucket-policy --bucket $BUCKET --query Policy --output text

```

Task 4 — Identity Policy vs Resource Policy

Cloud storage is governed by two policies at once: the **identity policy** attached to the caller and the **resource policy** attached to the bucket. When they disagree, an explicit Deny always wins. Prove it.

```

# An analyst whose IAM policy allows reading everything
aws $EP iam create-user --user-name DataAnalyst

cat > analyst-iam.json <<'JSON'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": "*"
  }]
}
JSON

aws $EP iam put-user-policy --user-name DataAnalyst \
  --policy-name S3ReadAll --policy-document file://analyst-iam.json

# Note both values - you will paste them into a named profile
aws $EP iam create-access-key --user-name DataAnalyst \
  --query 'AccessKey.[AccessKeyId,SecretAccessKey]' --output text

# Copy the two values into these variables - keep the quotes,
# and do not include angle brackets.
ANALYST_KEY_ID='PASTE_KEY_ID_HERE'
ANALYST_SECRET='PASTE_SECRET_HERE'

aws configure --profile analyst set aws_access_key_id "$ANALYST_KEY_ID"
aws configure --profile analyst set aws_secret_access_key "$ANALYST_SECRET"
aws configure --profile analyst set region us-east-1

```

The analyst's IAM policy says *read anything*. Now the bucket owner disagrees about one prefix:

```

cat > deny-confidential.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAnalystInternal",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::000000000000:user/DataAnalyst"},

```

```

    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::$BUCKET/internal/*"
  },
  {
    "Sid": "DenyAnalystConfidential",
    "Effect": "Deny",
    "Principal": {"AWS": "arn:aws:iam::000000000000:user/DataAnalyst"},
    "Action": "s3:*",
    "Resource": "arn:aws:s3:::$BUCKET/confidential/*"
  }
]
}
JSON

aws $EP s3api put-bucket-policy --bucket $BUCKET --policy file://deny-
confidential.json

# Should SUCCEED - allowed by both policies
AWS_PROFILE=analyst aws $EP s3api get-object \
  --bucket $BUCKET --key internal/roster.txt analyst-internal.txt && echo "internal:
ALLOWED"

# Should FAIL - IAM allows, but the bucket policy explicitly denies
AWS_PROFILE=analyst aws $EP s3api get-object \
  --bucket $BUCKET --key confidential/record.txt analyst-conf.txt || echo
"confidential: DENIED"

```

Verify or explain: If LocalStack was not started with `ENFORCE_IAM=1`, both calls will succeed. Restart the container with the flag and retry. If it still does not deny, record both policy documents as evidence and write out the evaluation logic yourself: default deny → any explicit Deny → any explicit Allow. State which statement decides each of the two requests.

Caution: Remove this policy before Session B: `aws $EP s3api delete-bucket-policy --bucket $BUCKET`. A Deny statement scoped to `s3:*` can lock **you** out as well if the principal ARN does not match exactly what you expected — which is itself one of the most common resource-policy incidents in production. If you do lock yourself out, delete the bucket policy or restart the LocalStack container.

Note: End of Session A. Keep your bucket, `$BUCKET` value, and all outputs — Session B builds directly on them.

Session B (Week 12) — Protecting, Retaining and Retiring Data

Task 5 — Default Encryption at Rest (SSE-KMS)

In Lab 3 you encrypted a file by hand. Here you make encryption a **property of the bucket**, so that every object is encrypted whether or not the developer who uploads it remembers to ask.

```

# A dedicated key for this bucket (the KMS concepts are from Lab 3)
export KEY_ID=$(aws $EP kms create-key \
  --description 'IKB42603 Lab6 patient records bucket key' \
  --query 'KeyMetadata.KeyId' --output text)

```

```

echo $KEY_ID

cat > encryption.json <<JSON
{
  "Rules": [{
    "ApplyServerSideEncryptionByDefault": {
      "SSEAlgorithm": "aws:kms",
      "KMSEMasterKeyID": "$KEY_ID"
    },
    "BucketKeyEnabled": true
  }]
}
JSON

aws $EP s3api put-bucket-encryption --bucket $BUCKET \
  --server-side-encryption-configuration file://encryption.json
aws $EP s3api get-bucket-encryption --bucket $BUCKET

# Upload with NO encryption flags at all - the bucket applies the key for you
aws $EP s3api put-object --bucket $BUCKET \
  --key confidential/record-v2.txt --body confidential-record.txt

aws $EP s3api head-object --bucket $BUCKET --key confidential/record-v2.txt \
  --query '[ServerSideEncryption,SSEKMSKeyId,BucketKeyEnabled]' --output text

```

The head-object output should report `aws:kms` and your key id. That is your evidence that a default control protected an object without the uploader doing anything.

Security tip: `BucketKeyEnabled: true` is the envelope-encryption optimisation from Lab 3 applied at bucket scale: one data key is reused across many objects instead of one KMS call per object. Confidentiality is unchanged; cost and latency fall sharply.

Task 6 — Delegated Access and the Condition-Key Trap

A presigned URL grants a specific action on a specific object for a limited time, to someone with no AWS identity at all. It is the correct answer to 'how do I share this file' — and it is frequently misused.

```

# Time-bounded, signed, single-object access
aws $EP s3 presign s3://$BUCKET/internal/roster.txt --expires-in 60

# Paste the URL into the variable - keep the quotes. The expiry test
# below must reuse the SAME url, which is why it goes in a variable.
URL='PASTE_PREIGNED_URL_HERE'

curl -s -w ' <-- HTTP %{http_code}\n' "$URL"

# Wait for it to lapse, then try the same url again
sleep 65
curl -s -o /dev/null -w 'after expiry: HTTP %{http_code}\n' "$URL"

```

Verify or explain: LocalStack may not enforce the expiry and can return HTTP 200 on the second call. If so, inspect the URL itself: identify the Expires / X-Amz-Expires and Signature

parameters, and explain in your report what each one binds and why anyone holding the URL before it lapses is fully authorised.

Now the trap. The following policy is copied verbatim from countless security hardening guides — it denies any request that did not arrive over TLS. Apply it and watch what happens to **every** subsequent command.

```
cat > secure-transport.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyUnencryptedTransport",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": ["arn:aws:s3:::$BUCKET", "arn:aws:s3:::$BUCKET/*"],
    "Condition": {"Bool": {"aws:SecureTransport": "false"}}
  }]
}
JSON

aws $EP s3api put-bucket-policy --bucket $BUCKET --policy file://secure-transport.json

# Any ordinary call - expect it to be refused
aws $EP s3api list-objects-v2 --bucket $BUCKET

# Recover before continuing
aws $EP s3api delete-bucket-policy --bucket $BUCKET
```

Caution: You have just locked yourself out of your own bucket, and the policy was correct. Your LocalStack endpoint is plain **http://**, so `aws:SecureTransport` is false for every request you make — the Deny matches all of them. On real AWS the endpoint is HTTPS, the condition evaluates to true, and the statement only catches genuinely insecure callers. In your report, explain why a condition key must always be evaluated against the environment it will run in, not the one it was written for.

Task 7 — Versioning, Delete Markers & Data Remanence

Lab 2 showed remanence inside a container volume. Object storage has its own version: with versioning enabled, delete does not delete. It writes a **delete marker** over the top and every prior version survives underneath.

```
aws $EP s3api put-bucket-versioning --bucket $BUCKET \
  --versioning-configuration Status=Enabled
aws $EP s3api get-bucket-versioning --bucket $BUCKET

# Two more revisions of the same record
echo 'Patient: Ahmad bin Ali, Diagnosis: hypertension' > rec-v2.txt
echo 'Patient: [REDACTED], Diagnosis: [REDACTED]' > rec-v3.txt

aws $EP s3api put-object --bucket $BUCKET --key confidential/record.txt \
  --body rec-v2.txt --query VersionId --output text
aws $EP s3api put-object --bucket $BUCKET --key confidential/record.txt \
```

```
--body rec-v3.txt --query VersionId --output text

aws $EP s3api list-object-versions --bucket $BUCKET \
  --prefix confidential/record.txt \
  --query 'Versions[][VersionId,IsLatest,Size]' --output table
```

The oldest entry is listed with version id null — that is the copy you uploaded in Task 1, before versioning existed. Now 'delete' the record and see what actually happens.

```
aws $EP s3api delete-object --bucket $BUCKET --key confidential/record.txt

# A delete marker is now the current version
aws $EP s3api list-object-versions --bucket $BUCKET \
  --prefix confidential/record.txt \
  --query 'DeleteMarkers[][VersionId,IsLatest]' --output table

# To an ordinary reader the object is gone
aws $EP s3api get-object --bucket $BUCKET --key confidential/record.txt gone.txt

# But the original, unredacted record is still there
aws $EP s3api get-object --bucket $BUCKET --key confidential/record.txt \
  --version-id null recovered.txt
cat recovered.txt
```

Caution: recovered.txt contains the original diagnosis — the data you redacted in v3 and then deleted. This is object-level data remanence, and it is why 'we deleted the record' is not an acceptable answer to a data-subject erasure request under a privacy regime such as the PDPA or GDPR. Removing it for real requires deleting **every version by id**:

```
# Permanent, per-version deletion
aws $EP s3api delete-object --bucket $BUCKET \
  --key confidential/record.txt --version-id null

aws $EP s3api list-object-versions --bucket $BUCKET \
  --prefix confidential/record.txt --query 'Versions[][VersionId,Size]' --output
table
```

Task 8 — Lifecycle, Retention & Cryptographic Erasure

Deleting versions by hand does not scale. A lifecycle configuration is the automated, auditable expression of your retention policy — and it is exactly the artefact an auditor will ask for in Week 11.

```
cat > lifecycle.json <<'JSON'
{
  "Rules": [
    {
      "ID": "RetireConfidentialRecords",
      "Filter": {"Prefix": "confidential/"},
      "Status": "Enabled",
      "Expiration": {"Days": 365},
      "NoncurrentVersionExpiration": {"NoncurrentDays": 30}
    },
    {
```

```

    "ID": "AbortIncompleteUploads",
    "Filter": {"Prefix": ""},
    "Status": "Enabled",
    "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
  }
]
}
JSON

```

```
aws $EP s3api put-bucket-lifecycle-configuration --bucket $BUCKET \
--lifecycle-configuration file://lifecycle.json
```

```
aws $EP s3api get-bucket-lifecycle-configuration --bucket $BUCKET \
--query 'Rules[][ID,Status]' --output table
```

Finally, the fastest deletion available in the cloud. Every object in this bucket is wrapped by one KMS key. Destroy the key and the ciphertext becomes unrecoverable noise, however many copies, versions or backups exist.

```
aws $EP kms describe-key --key-id $KEY_ID \
--query 'KeyMetadata.[KeyId,KeyState,Enabled]' --output text

aws $EP kms disable-key --key-id $KEY_ID
aws $EP kms schedule-key-deletion --key-id $KEY_ID --pending-window-in-days 7

aws $EP kms describe-key --key-id $KEY_ID \
--query 'KeyMetadata.[KeyState,DeletionDate]' --output text

# Attempt to read an object encrypted under the disabled key
aws $EP s3api get-object --bucket $BUCKET \
--key confidential/record-v2.txt after-erasure.txt
```

Verify or explain: LocalStack may still return the object because it does not always re-check key state on read. If your read succeeds, demonstrate the same principle at the KMS layer instead — repeat the Lab 3 Task 6 sequence (kms encrypt → disable-key → kms decrypt) and attach the failed decrypt. Then explain why cryptographic erasure gives an auditor a stronger assurance than overwriting, given that you do not control the physical media.

Deliverables & Assessment

1. Screenshots (label each clearly)

- The list-objects-v2 table and the classification tag on the confidential object (Task 1).
- The anonymous curl returning **HTTP 200** and the leaked record (Task 2).
- The get-public-access-block output showing all four flags true, plus the re-tested anonymous read (Task 3).
- The analyst's two attempts — internal allowed, confidential denied — or your written evaluation if not enforced (Task 4).
- The head-object output showing aws:kms and your key id (Task 5).
- The bucket-wide failure caused by the aws:SecureTransport policy (Task 6).

- The version listing, the delete marker, and `recovered.txt` still containing the original diagnosis (Task 7).
- The lifecycle rules table and the key state after `schedule-key-deletion` (Task 8).

2. Data Classification Table

The completed four-column table from Task 1, with the control you actually implemented named in the final column.

3. Short-Answer Questions

1. Which single element of the Task 2 policy caused the exposure, and why is `Principal: "*"` more dangerous on a bucket policy than an over-broad IAM policy attached to one user?
2. Explain the difference between an identity-based policy and a resource-based policy. In Task 4, which one decided each of the analyst's two requests?
3. Block Public Access is described as a guardrail rather than a control. What is the difference, and why does the distinction matter for an organisation with many engineers?
4. Your bucket has default SSE-KMS encryption. Does that protect the confidential record from the analyst in Task 4? Explain precisely what server-side encryption does and does not defend against.
5. A patient invokes their right to erasure. Using your Task 7 evidence, explain why `delete-object` alone is not compliant, and describe two mechanisms that would make the deletion provable.
6. You are the auditor in Week 11. Name three commands from this lab whose output you would collect as compliance evidence, and state which control each one evidences.

4. Verification Command

Paste the output of the following block to prove the bucket's final security posture:

```
echo "=== IKB42603 Lab 6 verification: $BUCKET ==="
aws $EP s3api get-public-access-block --bucket $BUCKET \
  --query 'PublicAccessBlockConfiguration' --output text
aws $EP s3api get-bucket-versioning --bucket $BUCKET --output text
aws $EP s3api get-bucket-encryption --bucket $BUCKET \
  --query
'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.[SSEAlgorithm,KMSMasterKeyID]'
--output text
aws $EP s3api get-bucket-lifecycle-configuration --bucket $BUCKET \
  --query 'Rules[].[ID,Status]' --output text
aws $EP kms describe-key --key-id $KEY_ID --query 'KeyMetadata.KeyState' --output text
```

Security Best-Practices Checklist

- ☐ Every object carries a classification tag before any access decision is made.
- ☐ No bucket policy names `Principal: "*" ; anonymous` access was tested and is refused.
- ☐ Block Public Access is enabled on all four flags.
- ☐ Access is granted by least privilege and scoped to a key prefix, never to `/*` by default.
- ☐ Default encryption at rest is `aws:kms` with a customer-managed key.
- ☐ Sharing uses time-bounded presigned URLs, not permanent public objects.

- ☐ Versioning is enabled, and the team understands that delete markers do not destroy data.
- ☐ A lifecycle configuration expresses the retention policy, and cryptographic erasure is available for provable deletion.

Cleanup & Teardown

A versioned bucket cannot be emptied with `s3 rb --force` — that command ignores non-current versions and delete markers, and the bucket deletion fails with `BucketNotEmpty`. This is the same lesson as Task 7, one last time: you must remove every version explicitly.

```
aws $EP s3api delete-bucket-policy --bucket $BUCKET

# Delete all object versions, then all delete markers
aws $EP s3api delete-objects --bucket $BUCKET --delete "$(aws $EP s3api \
  list-object-versions --bucket $BUCKET --output json \
  --query '{Objects: Versions[.]{Key:Key,VersionId:VersionId}}')"

aws $EP s3api delete-objects --bucket $BUCKET --delete "$(aws $EP s3api \
  list-object-versions --bucket $BUCKET --output json \
  --query '{Objects: DeleteMarkers[.]{Key:Key,VersionId:VersionId}}')"

# The bucket is only now genuinely empty
aws $EP s3api list-object-versions --bucket $BUCKET --output text
aws $EP s3api delete-bucket --bucket $BUCKET

aws $EP iam delete-user-policy --user-name DataAnalyst --policy-name S3ReadAll
aws $EP iam delete-user --user-name DataAnalyst

docker rm -f localstack
rm -f *.json *.txt
```

Expansion Ideas (Advanced Students)

- **Object Lock / WORM:** create a bucket with object lock enabled and apply a compliance-mode retention period, then attempt to delete a locked object — the evidence-integrity control behind Week 11 audit reporting.
- **Automated detection:** write a script that lists every bucket in the account and flags any without Block Public Access, default encryption or versioning — a miniature Cloud Security Posture Management tool.
- **Infrastructure as Code:** recreate the entire secure bucket (policy, encryption, versioning, lifecycle) in Terraform pointed at LocalStack, and scan the plan with Checkov before applying.
- **Event-driven response:** configure an S3 event notification to SNS or SQS on `s3:ObjectCreated:*` and log every upload — feeding the monitoring pipeline from Lab 5.
- **Client-side encryption:** encrypt an object with OpenSSL before upload and compare the threat model against SSE-KMS. Which one protects you from the cloud provider itself?

References

- Course lectures — Week 4 (Data Protection); Week 10 (Policy, Compliance & Risk); Week 11 (Compliance Assessment & Reporting).
- Amazon S3 security best practices — docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html
- Amazon S3 versioning and lifecycle — docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html
- LocalStack S3 coverage and limitations — docs.localstack.cloud/references/coverage
- CSA Security Guidance v5 — Domain 5, Data Security; and the Data Security Lifecycle.
- MCMC MTSFB TC G017:2021 — Information Security Requirements for Cloud Service Providers (data handling clauses).